

AI Agent 架构模式与 设计范式

从 ReAct 到 Multi-Agent, 从 MCP 到 A2A

2026.08 | 基于 20+ 行业来源的深度调研

Agent 不只是 **Prompt + LLM** 到底还需要什么？

一个能在生产环境可靠运行的 Agent，需要推理、编排、记忆、工具四层能力的系统性设计。
这不是一个模型选择问题，而是一个架构设计问题。

先看一个 Agent 到底由哪些层构成 →

Agent 架构的四层模型

思

推理层 Reasoning

LLM 核心：思考、决策、生成

排

编排层 Orchestration

控制流、状态管理、工作流图

忆

记忆层 Memory

工作记忆 / 情景 / 语义 / 程序记忆

具

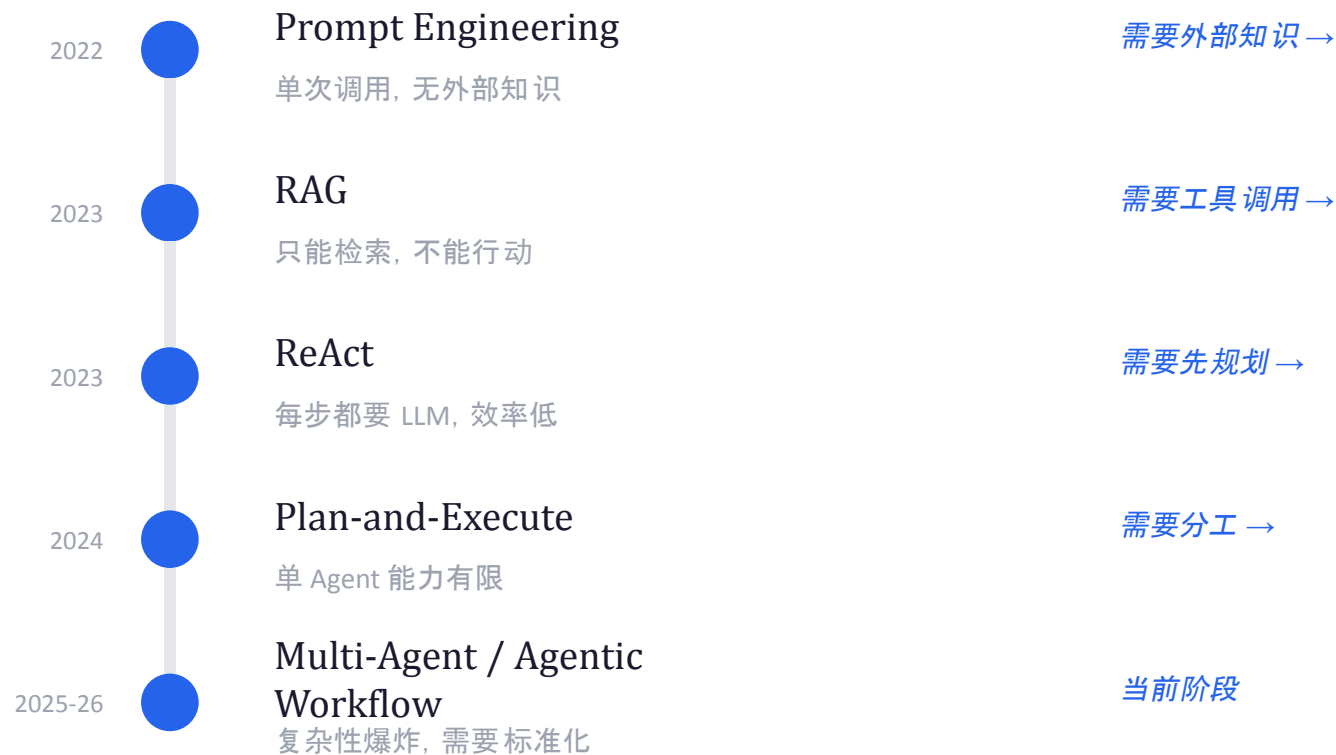
工具层 Tool Integration

函数调用、MCP 服务器、沙箱执行

这四层架构是怎么一步步演进出来的？ →

从 Prompt 到 Agentic Workflow

每一代模式的出现, 都是为了解决前一代的核心瓶颈:



了解了演进脉络, 来看当前主流的五大核心模式 →

五大核心 设计模式

ReAct

Think → Act → Observe

透明度高

Plan-Execute

先规划再执行

92% 完成率

Reflection

生成→批判→精炼

80→91% 准确率

Multi-Agent

顺序 / 并行 / 循环

专业化分工

Tool Use

函数调用+沙箱执行

连接真实世界

ReAct: 推理与行动交织



优势

- 透明度高, 每步可审计 (合规友好)
- 适应性强, 可根据观察调整策略
- 适用: 研究、客户支持、合规场景

劣势

- 每个循环需新 LLM 调用, 延迟高
- 可能陷入无效循环
- 成本随步骤数线性增长

能否先规划好所有步骤, 再一步步执行? → Plan-Execute →

Plan-Execute: 先规划, 再执行



92%
3.6x

任务完成率

加速 vs 顺序 ReAct 执行

成本优化策略

贵模型 (o3/Opus) 只做规划, 便宜模型 (Haiku/GPT-4o-mini) 执行具体步骤。规划一次, 执行多次。

Reflection: 自我批判与迭代精炼



🔄 迭代至满足质量标准

80% → 91%

HumanEval 编码基准提升

结合外部验证器(linter/编译器), 准确率可额外提升 30+ 个百分点。

适用场景: 代码生成、法律文书、文档写作 — 任何有明确质量标准可检验的任务。

Multi-Agent: 专业化分工协作

顺序 Sequential

$A \rightarrow B \rightarrow C$

如: 写作→编辑→校对

典型场景: 内容管道

并行 Parallel

$A / B / C$ 同时执行

管理者汇总结果

典型场景: 多维分析

循环 Loop

$A \leftrightarrow B$ 来回对话

如: 开发者↔审查者

典型场景: 代码审查

⚠ Princeton NLP 发现: 单 Agent 在 64% 的基准任务上匹配或超越多 Agent 系统。不要过早多 Agent 化。

有了多个 Agent, 请求该发给谁? → Routing 路由模式 →

Routing: 智能请求分发

在生产系统中, 路由通常是最先被实现的模式 — 它决定了请求"去哪里"。

条件路由

基于规则/关键词分发到不同 Agent

如: 订单查询→订单 Agent, 退款→退款 Agent

语义路由

基于意图理解自动分类

LLM 判断用户意图, 路由到对应专家

路由 + 编排

先路由到正确管道, 再在管道内编排

生产系统的标准组合模式

路由解决了"去哪里", 编排解决了"怎么执行" → 六大编排模式 →

六大生产级编排模式

Orchestrator-Worker

中央分解→委托→汇总

✓ 成本降 40-60%

✗ 单点故障

Fan-Out / Fan-In

并行执行后聚合

✓ 减 75% 时间

✗ 冲突二次方增长

Dynamic Handoff

自评后转交专家

✓ 适应性强

✗ 上下文丢失

Sequential Pipeline

线性链式执行

✓ 实现简单

✗ 错误级联

Multi-Agent Debate

Agent 互相挑战观点

✓ 减少幻觉

✗ 可能不收敛

Adaptive Planning

动态构建执行计划

✓ 发现式解决

✗ 目标漂移

按问题结构选编排模式

问题类型	推荐模式
已知分解	Orchestrator-Worker
固定线性步骤	Sequential Pipeline
独立并行工作	Fan-Out / Fan-In
需要质量验证	Multi-Agent Debate
不可预测路由	Dynamic Handoff
开放性问题	Adaptive Planning
实际系统很少单独使用一种模式 — 通常是路由 + 编排 + 反思的组合	

编排解决了"怎么跑", 但人在哪里介入? → Human-in-the-Loop →

Human-in-the-Loop 设计维度

企业 Agent 能否上生产, 往往取决于"人在哪里介入"的架构决策。

全自动

无需人工
信息检索、日志分析

业务风险: 低

半自动

Agent 执行, 人工抽检
内容生成、数据处理

业务风险: 中

人工审批门

关键步骤需人工确认
金融交易、法律文书

业务风险: 高

人工主导

Agent 辅助, 人做决策
医疗诊断、重大合同

业务风险: 极高

核心原则: 风险越高, 人工介入越深。审批门是架构设计的一部分, 不是事后补丁。

协议栈：MCP + A2A

MCP — Agent → 工具

Model Context Protocol

垂直互操作：连接工具、数据源、服务

9700万

月 SDK 下载量

5,800+

公开 MCP 服务器

所有主流提供商已采用

A2A — Agent → Agent

Agent-to-Agent Protocol

水平互操作：Agent 发现、通信、协作

Agent Cards

自动发现机制

完整任务状态机

submitted→working→completed/failed

100+ 企业加入支持

三层协议栈：WebMCP(结构化 Web 访问) → MCP (Agent→工具) → A2A (Agent→Agent)

Agent 怎么“记住”东西？→ 记忆架构 →

记忆架构:四种类型 + 生产栈

工作记忆

当前任务上下文

小型快速缓存

情景记忆

历史交互记录

pgvector

语义记忆

事实与知识

Postgres

程序记忆

学到的流程/技能

Neo4j / 图层

2026 最常见错误

把所有记忆当作一个大的向量数据库。向量搜索擅长语义相似度，但在多跳检索、时效性和精确召回上表现差。生产级记忆必须混合多种存储。

主流框架对比(2026 Q2)

框架	架构理念	学习曲线	控制力	适用场景
LangGraph	显式状态图+检查点	陡峭	最高	生产级有状态系统
CrewAI	角色扮演 Crew	最低	中等	快速原型/团队工作流
MS Agent Fw	对话式 Agent 团队	中等	中等	微软生态
Claude SDK	层级子 Agent 生成	低-中	高	Anthropic 生态
Pydantic AI	类型安全 Agent	低	中等	类型严格项目

2026 Q2 生态爆发: MS 合并 SK+AutoGen、Anthropic 添加子 Agent、CrewAI 可插拔记忆、Pydantic AI V2 稳定

框架只是工具 — 更重要的是避免踩坑 →

反模式与常见陷阱

✗ 过早多 Agent 化

64% 的任务单 Agent 就够了 (Princeton NLP)。多 Agent 增加的不是能力, 是复杂性。

✓ 先用单 Agent + 工具解决, 证明不够再拆分

✗ 过度工程

追求"最先进"的架构而非"最合适"的。生产系统 40% 多 Agent 试点 6 个月内失败。

✓ 从最简模式开始, 按需增加复杂性

✗ 忽视简单方案

并非所有自动化都需要 Agent。确定性流程用 workflow, 模式匹配用规则引擎。

✓ 问自己: 这个任务真的需要 LLM 推理吗?

✗ 忽视可观测性

Agent 的"成功"(HTTP 200)不等于正确。大多数 Agent 故障是静默的。

✓ 从第一天就建立轨迹级追踪

知道了该避什么, 那选型决策框架是什么? →

选型决策框架

先识别你的主要瓶颈, 再选对应的模式:

幻觉 / 准确性差



先加 Reflection

速度慢 / 成本高



切换到 Plan-Execute

Prompt 不可控 / 太长



拆分为 Multi-Agent

请求种类多



前置 Routing 层

高风险操作



加入 Human-in-the-Loop

需要连接外部系统



接入 MCP + Tool Use

黄金法则: 从最简单的能解决问题的模式开始, 在证明必要时再增加复杂性

总结

1

架构设计比模型选择更重要。四层架构(推理/编排/记忆/工具)是 Agent 的骨架, 模式选择决定了它怎么跑。

2

从最简单的模式开始。64% 的任务单 Agent 就够了。过早多 Agent 化是 2026 年最常见的架构错误。

3

MCP + A2A 正在成为行业标准。协议栈的成熟让 Agent 从"独狼"走向"生态", 但前提是你的基础架构先对了。